1. Well-Defined Problems and Solutions (8 Marks)

Definition

A well-defined problem is a problem in which all the components of problem formulation are clearly specified, so that a problem-solving agent can search for a solution systematically.

Five components of a well-defined problem

1. Initial State: The state from which the agent starts solving the problem.
2. Actions: The set of possible actions that the agent can perform in a given state.
3. Transition Model (Successor Function): Describes the result of performing an action in a particular state. It determines the next state reached after an action.
4. Goal Test: Checks whether the current state satisfies the goal condition.
5. Path Cost: Assigns a numerical cost to each path. Usually, it is the sum of the step costs of individual actions and is denoted by g.

Solutions

- Feasible Solution: A path from the initial state to a state satisfying the goal test.
- Optimal Solution: The solution having the lowest path cost among all possible solutions.

Example: Travelling from Arad to Bucharest

- Initial state: In(Arad)
- Actions: Go to Sibiu, Timisoara, or Zerind.
- Transition model: Specifies the city reached after each action.
- Goal test: Checks whether the agent has reached Bucharest.
- Path cost: Total distance travelled.

The path with the lowest total distance is the optimal solution.

Pseudocode of a Simple Problem-Solving Agent

```
function SIMPLE-PROBLEM-SOLVING-AGENT(p) returns an action
    inputs: p, a percept
    static: s, an action sequence, initially empty
            state, some description of the current world state
            g, a goal, initially null
            problem, a problem formulation

    state ← UPDATE-STATE(state, p)

    if s is empty then
        g ← FORMULATE-GOAL(state)
        problem ← FORMULATE-PROBLEM(state, g)
        s ← SEARCH(problem)

    action ← RECOMMENDATION(s, state)
    s ← REMAINDER(s, state)

    return action
```

Explanation of the pseudocode

1. Update State: The agent updates its current state using the received percept.
2. Check Action Sequence: If the sequence is empty, the agent must find a new solution.
3. Formulate Goal: The agent determines the goal to be achieved.
4. Formulate Problem: The agent defines the problem using the current state and goal.
5. Search: The search algorithm finds a sequence of actions to reach the goal.
6. Check Failure: If no solution is found, the agent returns a null action.
7. Select Action: `FIRST(seq)` selects the first action in the sequence.
8. Update Sequence and Return: `REST(seq)` removes the first action and retains the remaining actions. The selected action is then returned.

2. Explain the following terms with an example

A problem-solving agent uses the following components to define a problem clearly. Consider the example of travelling from Arad to Bucharest, as given in the AI notes.

a. Initial State

The initial state is the state in which the agent starts solving the problem.

Example: The agent starts its journey in Arad. Therefore, the initial state is `In(Arad)`.

b. Goal State

The goal state is the desired state that the agent wants to reach to solve the problem.

Example: The agent wants to reach Bucharest. Therefore, the goal state is `In(Bucharest)`.

c. Actions

Actions are the set of operations that an agent can perform in a particular state to move towards the goal.

Example: From Arad, the agent can choose to go to Sibiu, Timisoara, or Zerind.

$$Actions(Arad) = \{Go(Sibiu), Go(Timisoara), Go(Zerind)\}$$

d. Transition Model

The transition model describes the result of performing an action in a particular state. It specifies the next state reached after the action.

Example: If the agent is in Arad and performs the action `Go(Zerind)`, the resulting state is `In(Zerind)`.

$$RESULT(In(Arad), Go(Zerind)) = In(Zerind)$$

e. Path Cost

Path cost assigns a numerical cost to a path from the initial state to the goal state. It is usually the sum of the step costs of all actions along the path and is denoted by g.

Example: If the agent travels from Arad to Sibiu and then to Fagaras, the path cost is the sum of the distances of both road segments.

$$g(path) = c_1 + c_2$$

The path with the lowest total cost is considered the optimal solution when the objective is to minimize travel distance.

3. Explain the following problems with examples

a) Sliding-Block Puzzles (8-Puzzle Problem)

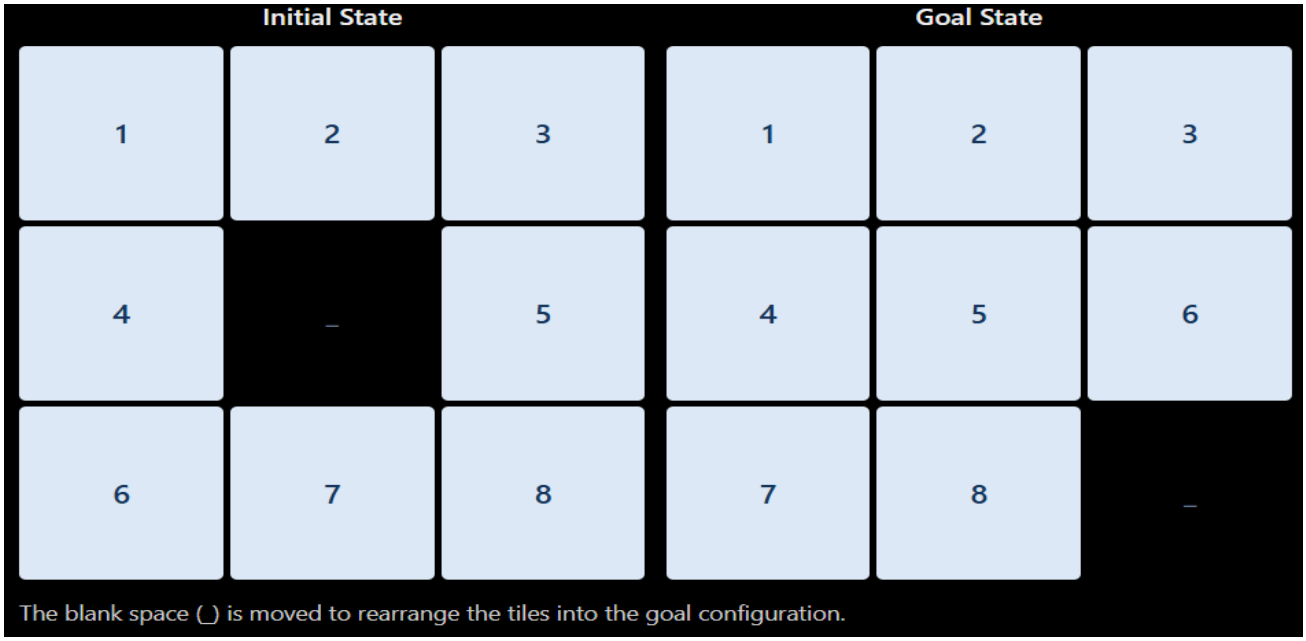
Definition: The sliding-block puzzle is a problem in which numbered tiles are arranged in a grid with one empty space. The objective is to move the tiles into a specified goal configuration by sliding them into the empty space.

Components of the problem:

- Initial State: Any random arrangement of the numbered tiles and the blank space.

- Actions: Move the blank space left, right, up, or down, whenever the move is valid.
- Transition Model: Moving the blank space swaps it with an adjacent numbered tile, producing a new state.
- Goal Test: Checks whether all the tiles are arranged in the required order.
- Path Cost: Each move costs 1; therefore, the path cost is the total number of moves.

Example:



The agent searches for a sequence of valid moves that transforms the initial arrangement into the goal arrangement with minimum path cost.

b) Traveling Salesperson Problem (TSP)

Definition: The Traveling Salesperson Problem (TSP) is a problem in which a salesperson must visit every city exactly once and return to the starting city while minimizing the total travel distance or cost.

Components of the problem:

- Initial State: The starting city from which the tour begins.
- Goal State: A complete tour visiting all cities exactly once and returning to the starting city.
- Intermediate States: Partial tours in which some cities have been visited.
- Actions/Operators: Moving from the current city to an unvisited city.
- Constraints: Each city must be visited exactly once, and the tour must start and end at the same city.
- Path Cost: The total distance or travel cost of the complete tour.

Example:

Consider four cities A, B, C and D. A salesperson starts at city A, visits all the other cities exactly once, and returns to A.

One possible tour is:

$$A \rightarrow B \rightarrow C \rightarrow D \rightarrow A$$

The total cost is the sum of the distances between consecutive cities. The objective is to find the tour with the minimum total cost.

Problem-solving methods: Dynamic programming, branch and bound, nearest-neighbour heuristic, and genetic algorithms.

c) Robot Navigation

Definition: Robot navigation is a problem in which a robot must find a path from its starting position to a target position while avoiding obstacles and satisfying movement constraints.

Components of the problem:

- Initial State: The robot's starting position (x_{start}, y_{start}), and possibly its orientation or velocity.
- Goal State: The target position (x_{goal}, y_{goal}), possibly including the required orientation.
- Intermediate States: The positions occupied by the robot as it moves through the environment.
- Actions/Operators: Moving up, down, left, right, or steering through continuous space.
- Constraints: Avoiding obstacles, staying within boundaries, and respecting movement limitations.
- Path Cost: A function that considers distance, time, energy consumption, smoothness, and other factors.

Example:

Consider a robot in a grid-based environment. It must move from a starting cell S to a destination cell G while avoiding blocked cells representing obstacles.

S → □ X □

□ → □ X □

X X □ → G

Here, `S` represents the starting position, `G` represents the goal, `X` represents an obstacle, and arrows indicate a possible route.

The robot searches for a valid path that reaches the goal while avoiding obstacles and minimizing the path cost.

Problem-solving methods: Breadth-First Search (BFS), A* search, and dynamic programming.

4. General Tree-Search and Graph-Search Algorithms

1. Tree-Search Algorithm

Informal description

Tree-search is a search algorithm that explores the search space as a tree, starting from the initial state and expanding nodes until the goal state is found.

The algorithm maintains a frontier, which contains the leaf nodes available for expansion.

Working:

- Initialize the frontier with the initial state.
- If the frontier is empty, return failure.
- Select and remove a leaf node from the frontier.
- If the node contains the goal state, return the solution.
- Otherwise, expand the node and add its child nodes to the frontier.
- Repeat until the goal is found or the frontier becomes empty.

Limitation: Tree-search does not remember previously explored states. Therefore, it may revisit states, repeat work, or enter an infinite loop when cycles exist.

Pseudocode

```
function TREE-SEARCH(problem) returns a solution, or failure
  initialize the frontier using the initial state of problem
  loop do
    if the frontier is empty then return failure
    choose a leaf node and remove it from the frontier
    if the node contains a goal state then return the corresponding solution
    expand the chosen node, adding the resulting nodes to the frontier
```

2. Graph-Search Algorithm

Informal description

Graph-search is an improved version of tree-search that avoids exploring the same state repeatedly. It maintains an explored set (closed list) to remember all expanded nodes.

Working:

- Initialize the frontier with the initial state and the explored set as empty.
- If the frontier is empty, return failure.
- Select and remove a node from the frontier.
- If the node satisfies the goal test, return the solution.
- Add the selected node to the explored set.
- Expand the node and generate its child nodes.
- Add only those child nodes that are neither in the frontier nor in the explored set.
- Repeat until the goal is found or the frontier becomes empty.

Advantages: It avoids repeated states, prevents many looping problems, and improves search efficiency.

Pseudocode

```
function GRAPH-SEARCH(problem) returns a solution, or failure
    initialize the frontier using the initial state of problem
    initialize the explored set to be empty
    loop do
        if the frontier is empty then return failure
        choose a leaf node and remove it from the frontier
        if the node contains a goal state then return the corresponding solution
        add the node to the explored set
        expand the chosen node, adding the resulting nodes to the frontier
            only if not in the frontier or explored set
```

Difference between Tree-Search and Graph-Search

Tree-Search	Graph-Search
Does not maintain an explored set.	Maintains an explored set.
May revisit the same state.	Avoids revisiting explored states.
Can get stuck in cycles.	Prevents repeated-state loops in typical finite searches.
Usually requires less memory.	Requires additional memory for explored states.

5. Measuring Problem-Solving Performance

The performance of a search algorithm is evaluated using the following four criteria.

1. Completeness

Completeness determines whether the search strategy is guaranteed to find a solution if a solution exists.

Example: Breadth-First Search (BFS) is complete when the branching factor is finite.

2. Optimality

Optimality determines whether the strategy finds the highest-quality or least-cost solution when several solutions exist.

Example: BFS is optimal when every step has the same cost.

3. Time Complexity

Time complexity measures how long it takes to find a solution. In AI search algorithms, it is measured by the number of nodes generated during the search.

4. Space Complexity

Space complexity measures how much memory is required to perform the search. It is measured by the maximum number of nodes stored in memory.

Parameters used to express complexity

- b — Branching factor: Maximum number of successors of any node.
- d — Depth of the shallowest goal node: The depth at which the nearest goal is found.
- m — Maximum depth: Maximum length of any path in the state space.

Total cost and trade-off

To evaluate the effectiveness of a search algorithm, we consider the total cost:

$$TotalCost = PathCost(g) + SearchCost$$

Here, search cost refers to the time required to find the solution.

There is a trade-off between spending more time to find an optimal solution with the lowest path cost and finding a solution faster with a slightly higher path cost.

6. a) Using Figure 2.1, formulate the route-finding problem from Arad to Bucharest. Identify the initial state, goal test, actions and path cost.
   b) Using Figure 2.1, give one possible solution path and calculate its path cost using the shown edge costs.
   c) Using Figure 2.1, identify an example of a repeated state or redundant path that could arise during search.

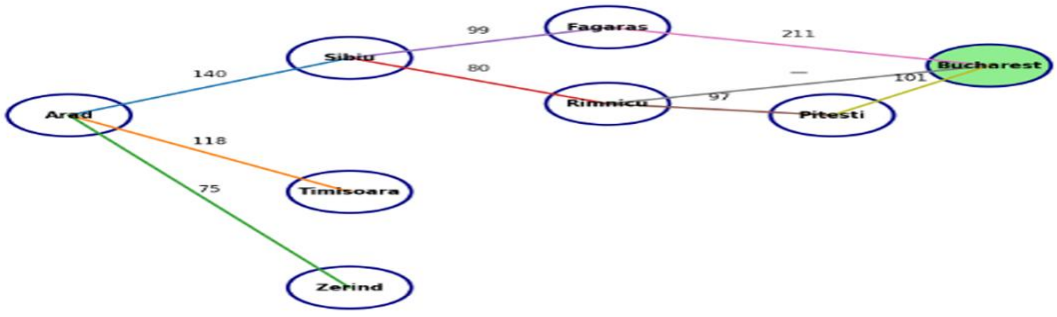


Figure 2.1

6. Route-Finding Problem: Arad to Bucharest

a) Formulate the route-finding problem from Arad to Bucharest

The route-finding problem can be formulated using the following components:

Initial State:

The agent starts at the city of Arad.

$$InitialState = In(Arad)$$

Goal Test:

The goal test checks whether the agent has reached the destination city, Bucharest.

$$GoalTest = In(Bucharest)$$

Actions:

The agent can travel from the current city to any directly connected neighbouring city.

From Arad, the available actions are:

- Go to Sibiu
- Go to Timisoara
- Go to Zerind

Path Cost:

The path cost is the total distance travelled along the selected route. It is calculated by adding the edge costs between consecutive cities in Figure 2.1.

$$g(path) = \sum \quad edgecosts$$

b) Give one possible solution path and calculate its path cost

One possible solution path from Arad to Bucharest is:

$$\text{Arad} \rightarrow \text{Sibiu} \rightarrow \text{Fagaras} \rightarrow \text{Bucharest}$$

From Figure 2.1, the edge costs are:

- Arad to Sibiu = 140
- Sibiu to Fagaras = 99
- Fagaras to Bucharest = 211

Total path cost:

$$g(\text{path}) = 140 + 99 + 211$$
$$= \boxed{450}$$

Therefore, the total path cost of this possible solution is 450 distance units.

c) Identify an example of a repeated state or redundant path

A repeated state occurs when a search algorithm reaches a city that has already been visited through another path.

Example: Consider the following paths between Arad and Sibiu:

$$\text{Arad} \longrightarrow \text{Sibiu}$$

and

$$\text{Arad} \longrightarrow \text{Sibiu} \longrightarrow \text{Arad} \longrightarrow \text{Sibiu}$$

In the second path, the cities Arad and Sibiu are revisited, creating a redundant path.

Another example is reaching Bucharest through Fagaras or through Pitesti. Both paths reach the same goal state, but their path costs may differ.

Conclusion: Graph-search can avoid repeatedly exploring already visited states by maintaining an explored set, thereby reducing redundant work during the search.

7. Discuss the Vacuum World, 8-Puzzle, and 8-Queens as Toy Problems. Compare Their State Representations and Action Models.

Toy problems are simplified problems used in Artificial Intelligence (AI) to study and test problem-solving and search algorithms.

a) Vacuum World

Definition: The Vacuum World is a toy problem in which a vacuum-cleaning agent must clean dirty squares in an environment.

State representation:

- The agent's location (left or right).
- Whether each square is clean or dirty.
- For two squares, there are $2 \times 2 \times 2 = 8$ possible states.

Action model:

The agent has three possible actions:

- Left: Move to the left square.
- Right: Move to the right square.
- Suck: Clean the current square.

Goal test: Checks whether all squares are clean.

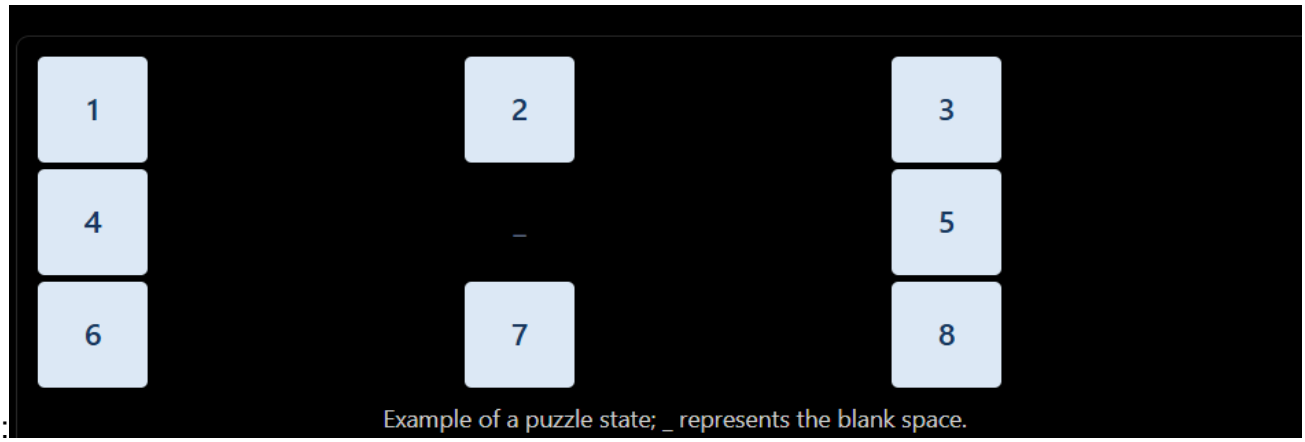
Path cost: Each step costs 1, so the path cost is the total number of steps taken.

b) 8-Puzzle

Definition: The 8-puzzle is a sliding-tile puzzle consisting of eight numbered tiles and one blank space arranged in a 3×3 grid. The objective is to arrange the tiles in the required goal configuration.

State representation:

A state specifies the location of each of the eight tiles and the blank space in the nine squares.

Example: 

Example of a puzzle state; _ represents the blank space.

Example of a puzzle state; _ represents the blank space.

Action model:

- Move the blank space left.
- Move the blank space right.
- Move the blank space up.
- Move the blank space down.

Only legal moves are allowed. Moving the blank space swaps it with an adjacent numbered tile.

Goal test: Checks whether the current state matches the required goal configuration.

Path cost: Each move costs 1, so the path cost is the total number of moves.

c) 8-Queens Problem

Definition: The 8-Queens problem involves placing eight queens on an 8×8 chessboard so that no queen attacks another queen.

State representation:

A state represents an arrangement of queens on the chessboard. In the complete-state formulation, states contain arrangements of zero to eight queens, with one queen per column in the leftmost columns and no queen attacking another.

Action model:

- Add a queen to any square in the leftmost empty column, provided it is not attacked by any existing queen.
- Alternatively, in a complete-state formulation, move an attacked queen to a safe row.

Goal test: Checks whether all eight queens are placed on the board and no queen attacks another.

Path cost: Path cost is generally not important because the objective is to find a valid arrangement rather than minimize the number of moves.

Comparison of State Representations and Action Models

Feature	Vacuum World	8-Puzzle	8-Queens
State representation	Agent location and dirt status of each square	Positions of eight tiles and blank	Arrangement of queens on the chessboard
Initial state	Any valid world state	Random arrangement of tiles	Empty board in incremental formulation
Actions	Left, Right, Suck	Move blank Left, Right, Up, Down	Place a queen in a safe square
Transition model	Moves the agent or cleans a square	Swaps the blank with an adjacent tile	Adds a queen to a safe square

Goal test	All squares are clean	Tiles match the goal arrangement	Eight queens placed without attacking one another
Path cost	Number of steps	Number of moves	Usually not important

8. Explain how the incremental formulation can reduce the search space by prohibiting placement of a queen in an attacked square.

The 8-Queens problem is an AI toy problem in which eight queens must be placed on an 8×8 chessboard so that no queen attacks another queen.

Incremental formulation

In the incremental formulation, the board starts with no queens, and queens are added one at a time.

- Initial state: An empty chessboard.
- States: Any arrangement of zero to eight queens on the board.
- Actions: Add a queen to any empty square.
- Transition model: Returns the board with a queen added to the specified square.
- Goal test: Checks whether eight queens are on the board and none attacks another.

How it reduces the search space

- Initially, there are 64 squares available for placing the first queen.
- Each new queen is placed only in a square that is not attacked by any previously placed queen.
- A queen attacks other queens in the same row, column, or diagonal. Therefore, attacked squares are excluded from further consideration.
- This prevents the algorithm from generating invalid arrangements and reduces the number of states that need to be explored.

In the basic incremental formulation, the number of possible sequences is approximately

$$64 \times 63 \times \cdots \times 57 \approx 10^{14}$$

By prohibiting placement in attacked squares, the search space is reduced significantly because invalid arrangements are eliminated early.

9. Breadth-First Search (BFS)

Definition

Breadth-First Search (BFS) is an uninformed search algorithm that explores the shallowest nodes first. It expands all nodes at the current depth level before moving to the next level.

BFS is commonly used to find the shortest path in an unweighted graph.

1. Data Structure Used

BFS uses a FIFO (First-In, First-Out) queue to store nodes that have been generated but not yet explored.

- Enqueue: Adds a node to the rear of the queue.
- Dequeue: Removes a node from the front of the queue.
- Explored set: Stores already visited nodes to avoid revisiting them in graph search.

2. Working of BFS

1. Insert the initial node into the queue.
2. If the queue is empty, return failure.
3. Remove the first node from the queue.
4. If it is the goal node, return the solution.
5. Mark the node as explored.
6. Generate its successor nodes and add unexplored successors to the queue.
7. Repeat until the goal is found or the queue becomes empty.

3. Algorithm / Pseudocode

```
function BREADTH-FIRST-SEARCH(problem) returns a solution, or failure
  node ← a node with STATE = problem.INITIAL-STATE, PATH-COST = 0
  if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
  frontier ← a FIFO queue with node as the only element
  explored ← an empty set
  loop do
    if EMPTY?(frontier) then return failure
    node ← POP(frontier)  /* chooses the shallowest node in frontier */
    add node.STATE to explored
    for each action in problem.ACTIONS(node.STATE) do
      child ← CHILD-NODE(problem, node, action)
      if child.STATE is not in explored or frontier then
        if problem.GOAL-TEST(child.STATE) then return SOLUTION(child)
        frontier ← INSERT(child, frontier)
```

4.

Completeness

BFS is complete if the branching factor b is finite. It is guaranteed to find a solution if one exists at a finite depth.

5. Optimality Condition

BFS is optimal when every step has the same cost (for example, each step costs 1). It finds the shallowest solution, which is also the least-cost solution under this condition.

6. Time Complexity

The time complexity of BFS is:

$$O(b^{d+1})$$

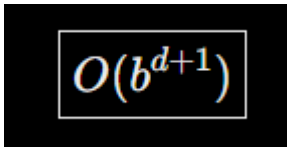
where:

- b = branching factor.
- d = depth of the shallowest goal node.

BFS may generate all nodes up to the next level before finding the goal.

7. Space Complexity

The space complexity of BFS is:



BFS stores the frontier and generated nodes in memory. Therefore, its memory requirement can become very large when the branching factor or solution depth increases.

Summary

Property	Breadth-First Search
Search strategy	Shallowest node first
Data structure	FIFO queue
Complete	Yes, if branching factor is finite
Optimal	Yes, if all step costs are equal
Time complexity	$O(b^{d+1})$
Space complexity	$O(b^{d+1})$

10. Differentiate between Breadth-First Search (BFS) and Depth-First Search (DFS)

Feature	Breadth-First Search (BFS)	Depth-First Search (DFS)

1. Search strategy	Expands the shallowest nodes first.	Expands the deepest nodes first.
2. Data structure	Uses a FIFO queue.	Uses a LIFO stack or recursion.
3. Node expansion	Explores all nodes at the current level before moving to the next level.	Explores one branch as deeply as possible before backtracking.
4. Completeness	Complete if the branching factor is finite.	Not complete in infinite-depth spaces or spaces with loops; complete in finite spaces with appropriate cycle handling.
5. Optimality	Optimal when all step costs are equal.	Not optimal in general.
6. Time complexity	$O(b^{d+1})$	$O(b^m)$
7. Space complexity	$O(b^{d+1})$	$O(bm)$
8. Memory usage	Requires more memory because it stores many nodes at each level.	Generally requires less memory because it stores the current path and remaining alternatives.
9. Suitable for	Finding the shortest path in an unweighted graph.	Exploring deep search spaces when memory is limited.
10. Backtracking	Does not normally use backtracking.	Uses backtracking when a branch reaches a dead end.

Notation:

- b = branching factor.
- d = depth of the shallowest goal node.
- m = maximum depth of the search tree.

11. Explain the Following Searching Techniques

a) Uniform-Cost Search (UCS)

Definition: Uniform-Cost Search is an uninformed search technique that expands the node with the lowest cumulative path cost $g(n)$. It uses a priority queue to select the least-cost node.

Working

1. Insert the initial node into a priority queue with cost 0.
2. Remove the node with the lowest path cost.
3. If it is the goal node, return the solution.
4. Otherwise, expand the node and calculate the path costs of its successors.
5. Insert the successors into the priority queue with their updated costs.
6. Repeat until the goal is found or the queue becomes empty.

Example

Suppose there are two paths to a goal:

- Path A: $S \to A \to G$, cost $2 + 8 = 10$
- Path B: $S \to B \to G$, cost $3 + 4 = 7$

UCS selects the path with cost 7, provided it is the least-cost solution.

Properties

- Completeness: Yes, if every step cost is at least some positive constant ε.
- Optimality: Yes, for non-negative step costs when the goal is tested upon removal from the priority queue.
- Time complexity: \(O\left(b^{\lceil C^*/\varepsilon\rceil}\right)\).
- Space complexity: \(O\left(b^{\lceil C^*/\varepsilon\rceil}\right)\).

Here, b is the branching factor, C^* is the optimal solution cost, and ε is the minimum positive step cost.

b) Depth-Limited Search (DLS)

Definition: Depth-Limited Search is a variation of Depth-First Search (DFS) in which a fixed depth limit L is imposed. The search does not expand nodes beyond this limit.

Working

1. Start from the initial node.
2. Explore nodes depth-first.
3. If the goal is found, return the solution.
4. If the depth limit is reached, stop expanding that branch.
5. Backtrack and explore other branches within the limit.
6. If no solution is found, return failure or a cutoff indication.

Example

Suppose the goal node is at depth 4, but the depth limit is set to 3. DLS cannot reach the goal and returns a cutoff. If the limit is increased to 4, the goal can be found.

Properties

- Completeness: Incomplete if $L < d$, where d is the depth of the shallowest goal.
- Optimality: Not optimal in general.
- Time complexity: $O(b^L)$.
- Space complexity: $O(bL)$.

Here, b is the branching factor and L is the depth limit.

c) Iterative Deepening Depth-First Search (IDDFS)

Definition: Iterative Deepening Depth-First Search combines the advantages of BFS and DFS. It repeatedly performs depth-limited searches, increasing the depth limit by one each time until a solution is found.

Working

1. Start with a depth limit of 0.
2. Perform DLS with the current limit.
3. If the goal is found, return the solution.
4. Otherwise, increase the depth limit by 1.
5. Repeat until the solution is found.

Example

If the goal is at depth 3, IDDFS searches with limits 0, 1, 2, and 3. It finds the goal when the limit reaches 3.

Properties

- Completeness: Yes, if the branching factor is finite and a solution exists at finite depth.
- Optimality: Yes, when all step costs are equal.
- Time complexity: $O(b^d)$.
- Space complexity: $O(bd)$.

Here, b is the branching factor and d is the depth of the shallowest goal.

Advantage: IDDFS combines the completeness of BFS with the low memory usage of DFS. It is useful when the solution depth is unknown.

d) Bidirectional Search

Definition: Bidirectional Search performs two simultaneous searches: one forward from the initial state and another backward from the goal state. The search stops when the two searches meet.

Working

1. Start one search from the initial state.
2. Start another search from the goal state.
3. Expand nodes from both directions.
4. Check whether the frontiers intersect.
5. When a common state is found, combine the two paths to obtain the solution.

Example

To find a route from Arad to Bucharest, one search proceeds from Arad while another proceeds backward from Bucharest. When both searches reach a common city, their paths can be combined into a complete route.

Properties

- Completeness: Yes, under finite branching and suitable search conditions.
- Optimality: Yes, when both directions use BFS and all step costs are equal.
- Time complexity: $O(b^{d/2})$.
- Space complexity: $O(b^{d/2})$.

Here, b is the branching factor and d is the depth of the solution.

Limitation: Bidirectional Search requires the goal state to be known and the backward search to be feasible.

12. Depth-First Search (DFS)

Definition

Depth-First Search (DFS) is an uninformed search technique that expands the deepest unexpanded node first. It explores one branch of the search tree as deeply as possible before backtracking to explore other branches.

1. Data Structure Used

DFS uses a LIFO (Last-In, First-Out) stack or recursion.

- Push: Adds a node to the top of the stack.
- Pop: Removes the top node from the stack.
- The most recently added node is explored first.
- In graph search, an explored set can be used to avoid revisiting states.

2. Working of DFS

1. Start from the initial node.
2. Check whether the node is the goal.
3. If it is not the goal, expand it and select a successor.
4. Continue exploring the deepest branch.
5. If a dead end is reached, backtrack to the previous node.
6. Explore another unexplored branch.
7. Repeat until the goal is found or no more nodes remain.

3. Completeness Issues

DFS is not complete in infinite-depth spaces or spaces containing loops because it may continue exploring an infinite path without returning to explore other branches.

However, DFS is complete in finite search spaces when repeated states and cycles are handled appropriately.

4. Optimality

DFS is not optimal because it does not guarantee that the solution found has the minimum path cost or is the shallowest solution.

It may find a longer solution before discovering a shorter one.

5. Time Complexity

The time complexity of DFS is:

$$\boxed{O(b^m)}$$

where:

- b = branching factor, the maximum number of children of any node.
- m = maximum depth of the search tree.

DFS may explore the entire search tree in the worst case.

6. Space Complexity

The space complexity of DFS is:

$$\boxed{O(bm)}$$

DFS requires relatively little memory because it stores the current path and the unexplored alternatives, rather than keeping every node in memory.

7. Backtracking Search

Backtracking search is a variation of DFS in which only one successor is generated at a time, rather than generating and storing all successors.

Working:

1. Start from the initial state.
2. Generate one successor and explore it.
3. Continue until the goal is found or a dead end is reached.
4. If a dead end is reached, return to the previous state.
5. Generate another successor and continue the search.

Advantages:

- Requires less memory than standard DFS.
- Avoids storing all successor nodes simultaneously.
- Useful for problems such as the 8-Queens problem and maze solving.

13. Determine the Uniform Least-Cost Path from S to G (Figure 2.2)

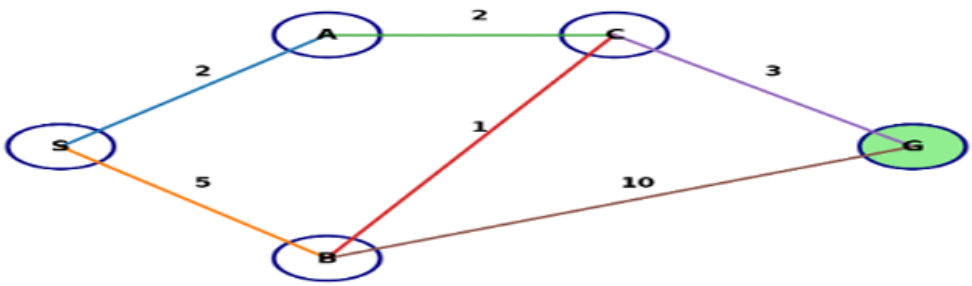


Figure 2.2

Uniform-Cost Search (UCS) expands the node with the lowest cumulative path cost $g(n)$ first. It uses a priority queue ordered by cumulative cost.

Step 1: Identify the edge costs

From Figure 2.2, the given edge costs are:

Edge	Cost

S → A	2
S → B	5
A → C	2
B → C	1
C → G	3
B → G	10

Step 2: Initialize the priority queue

Start from node S with path cost 0.

Node	Cumulative cost
S	0

Remove the node with the lowest cost and expand it.

Step 3: Expand S

From S, the available successors are A and B.

- Cost to A = $0 + 2 = 2$
- Cost to B = $0 + 5 = 5$

Priority queue:

Node	Cumulative cost

A	2
B	5

Choose A, as it has the lowest cumulative cost.

Step 4: Expand A

From A, the successor is C.

- Cost to C = $2 + 2 = 4$

Priority queue:

Node	Cumulative cost
C	4
B	5

Choose C, as its cost is lower than B's.

Step 5: Expand C

From C, the successors are B and G.

- Cost to B through C = $4 + 1 = 5$
- Cost to G through C = $4 + 3 = 7$

Priority queue:

Node	Cumulative cost
B	5

G	7

Choose B, as it has the lowest cost. Since B was already available at cost 5, its cost remains 5.

Step 6: Expand B

From B, the possible successors are C and G.

- Cost to C through B = $5 + 1 = 6$. This is higher than the existing cost of 4, so it is discarded.
- Cost to G through B = $5 + 10 = 15$. This is higher than the existing cost of 7, so it is discarded.

The next node selected is G, with cost 7.

Step 7: Goal Reached

Since G is the goal node, UCS terminates and returns the least-cost path.

$$\boxed{S \to A \to C \to G}$$

Total path cost:

$$g(G) = 2 + 2 + 3$$
$$= \boxed{7}$$

Final answer: The uniform least-cost path from S to ↓ S → A → C → G, with a total path cost of 7.

14. Determine the uniform least cost path from S to G and calculate its total path cost for the figure 2.3

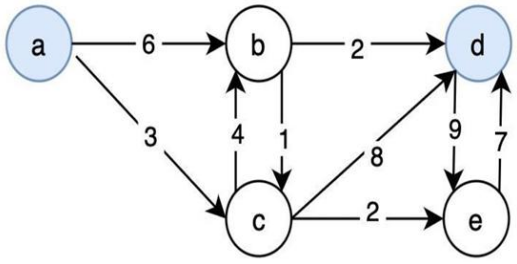


Figure -2.3: Here Initial Node =a and Goal Node = d

14. Determine the Uniform Least-Cost Path from S to G and Calculate Its Total Path Cost (Figure 2.3)

Uniform-Cost Search (UCS) expands the node with the lowest cumulative path cost first. It uses a priority queue to select the node with the minimum cost.

Step 1: Identify the edge costs

According to Figure 2.3, the initial node is a and the goal node is d.

Directed edge	Cost
$a \to b$	6
$a \to c$	3
$c \to b$	4
$b \to c$	1
$b \to d$	2
$c \to d$	8
$c \to e$	2
$d \to e$	9
$e \to d$	7

Step 2: Initialize the priority queue

Start from node a with cumulative path cost 0.

Node	Cumulative cost
a	0

Step 3: Expand node a

The successors of a are b and c.

- Cost to $b = 0 + 6 = 6$
- Cost to $c = 0 + 3 = 3$

Priority queue (lowest cost first):

Node	Cumulative cost
c	3
b	6

Choose c because it has the lowest cost.

Step 4: Expand node c

The successors of c are b, d, and e.

- Cost to b through $c = 3 + 4 = 7$. The existing cost to b is 6, so retain 6.
- Cost to $d = 3 + 8 = 11$
- Cost to $e = 3 + 2 = 5$

Priority queue:

Node	Cumulative cost
e	5
b	6
d	11

Choose e because it has the lowest cost.

Step 5: Expand node e

The successor of e is d.

- Cost to d through $e = 5 + 7 = 12$.
- The existing cost to d is 11, so retain 11.

Priority queue:

Node	Cumulative cost
b	6
d	11

Choose b because it has the lowest cost.

Step 6: Expand node b

The successors of b are c and d.

- Cost to c through $b = 6 + 1 = 7$. The existing cost to c is 3, so discard this path.
- Cost to d through $b = 6 + 2 = 8$. This improves the existing cost of 11 to 8.

Priority queue:

Node	Cumulative cost
d	8

Step 7: Goal reached

Remove d from the priority queue. Since d is the goal node, terminate the search.

The least-cost path is:

$$\boxed{a \to b \to d}$$

Total path cost:

$$\begin{aligned} g(d) &= 6 + 2 \\ &= \boxed{8} \end{aligned}$$

Final answer: The uniform least-cost path is $a \to b \to d$, and its total path cost is **8**.

15. Explain the Working of Depth-Limited Search with the Help of an Algorithm

Definition

Depth-Limited Search (DLS) is an uninformed search technique and a variation of Depth-First Search (DFS). It explores nodes depth-first but imposes a fixed depth limit L, beyond which no further nodes are expanded.

Working of Depth-Limited Search

1. Start from the initial state with a specified depth limit L.
2. Check whether the current state satisfies the goal test.
3. If the goal is found, return the solution.
4. If the depth limit reaches zero, return a cutoff, indicating that further exploration is not allowed.
5. Otherwise, expand the current state and explore its successors recursively with the depth limit decreased by one.
6. If a successor returns a solution, return that solution.
7. If all successors fail, return failure.

Algorithm of Depth-Limited Search

```
RECURSIVE-DLS(state, limit)
  if GOAL-TEST(state) then
    return solution
  else if limit = 0 then
    return cutoff
```

```
else
    cutoff_occurred ← false
    for each action in ACTIONS(state) do
        child ← RESULT(state, action)
        result ← RECURSIVE-DLS(child, limit - 1)
        if result = cutoff then
            cutoff_occurred ← true
        else if result ≠ failure then
            return result
    if cutoff_occurred then
        return cutoff
    else
        return failure
```

Example

Suppose the goal node is at depth 3 and the depth limit is $L = 2$. The algorithm explores only up to depth 2 and returns a cutoff because the goal lies beyond the limit. If the limit is increased to 3, the goal can be found.

Properties of DLS

- Completeness: Incomplete if the depth limit L is less than the depth d of the shallowest goal.
- Optimality: Not optimal in general because it may find a deeper solution before a shallower one.
- Time complexity: $O(b^L)$.
- Space complexity: $O(bL)$.

Here, b is the branching factor and L is the depth limit.

Conclusion: Depth-Limited Search prevents DFS from exploring infinitely deep paths by imposing a fixed depth limit. It is useful when the approximate depth of the solution is known, but a limit that is too small may prevent the algorithm from finding an existing solution.

16. Explain the Working of Iterative Deepening Search with a Suitable Search Tree

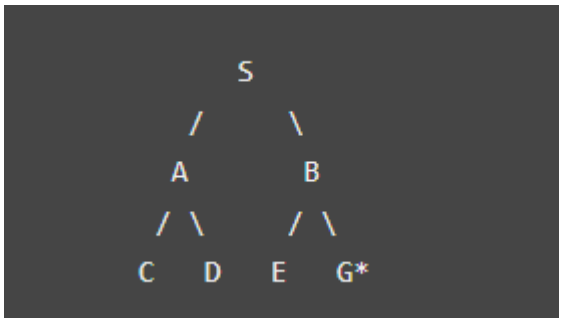
Definition

Iterative Deepening Search (IDS) is an uninformed search technique that combines the advantages of Breadth-First Search (BFS) and Depth-First Search (DFS). It repeatedly performs Depth-Limited Search (DLS), increasing the depth limit by one at each iteration until the goal is found.

Working of Iterative Deepening Search

1. Start with a depth limit of $L = 0$.
2. Perform Depth-Limited Search with the current limit.
3. If the goal is found, return the solution.
4. If the goal is not found, increase the depth limit by one.
5. Repeat the search until the goal is found.

Search Tree Example



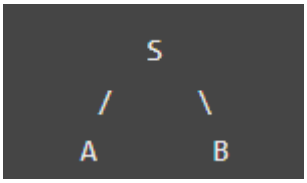
Iterations of IDS

Iteration 1: Depth limit = 0

S

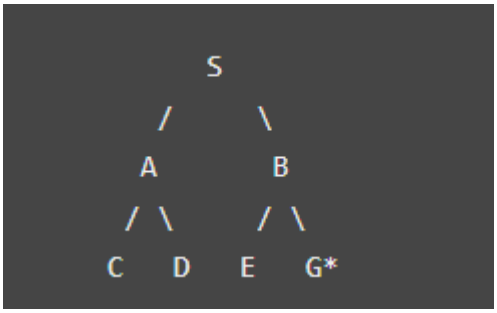
Only the root node S is explored. The goal is not found.

Iteration 2: Depth limit = 1



The nodes S, A, B are explored. The goal is not found.

Iteration 3: Depth limit = 2



The algorithm explores the tree depth-first, from left to right:

$$S \rightarrow A \rightarrow C \rightarrow D \rightarrow B \rightarrow E \rightarrow G$$

When node G is reached, the goal is found and the search terminates.

Algorithm

ITERATIVE-DEEPENING-SEARCH(problem)

for limit ← 0 to infinity do

result ← DEPTH-LIMITED-SEARCH(problem, limit)

if result ≠ cutoff then

return result

Properties of IDS

- Completeness: Complete if the branching factor is finite and a solution exists at a finite depth.
- Optimality: Optimal when all step costs are equal, because it finds the shallowest goal.
- Time complexity: $O(b^d)$.
- Space complexity: $O(bd)$.

Here, b is the branching factor and d is the depth of the shallowest goal node.

Advantages

- Combines the completeness of BFS with the memory efficiency of DFS.
- Does not require prior knowledge of the solution depth.
- Uses less memory than BFS because it explores depth-first.

17. Explain Bidirectional Search with a Suitable Diagram

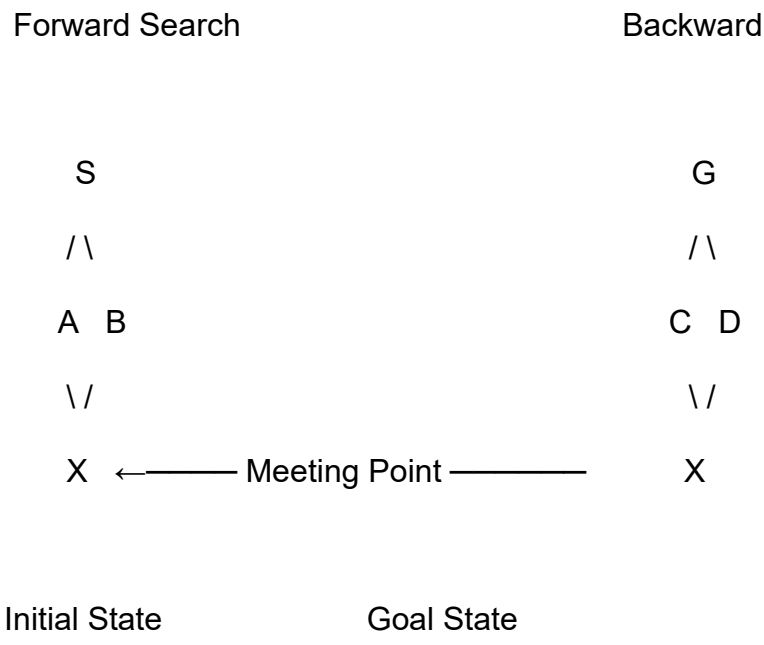
Definition

Bidirectional Search is an uninformed search technique that performs two simultaneous searches: one forward from the initial state and another backward from the goal state. The search terminates when the two searches meet at a common state.

Working of Bidirectional Search

1. Start the forward search from the initial state.
2. Start the backward search from the goal state.
3. Expand nodes in both directions.
4. Check whether the two search frontiers have a common state.
5. When a common state is found, combine the forward and backward paths to obtain the complete solution.

Suitable Diagram



The forward search starts at S, while the backward search starts at G. When both searches reach the common state X, their paths are joined to form the solution.

Example

Consider finding a path from Arad to Bucharest.

- Forward search: Arad → Sibiu → Fagaras
- Backward search: Bucharest → Fagaras
- Meeting point: Fagaras

The complete path is:

$$\text{Arad} \rightarrow \text{Sibiu} \rightarrow \text{Fagaras} \rightarrow \text{Bucharest}$$

Time and Space Complexity

Let b be the branching factor and d be the depth of the solution.

- Time complexity: $O(b^{d/2})$
- Space complexity: $O(b^{d/2})$

Instead of searching to depth d from one direction, each search explores approximately half the depth. This can significantly reduce the number of nodes explored.

Advantages

- Reduces search time compared with searching from only one direction.
- Can find the shortest path when both searches use BFS and all step costs are equal.
- Useful when the initial and goal states are known.

Limitations

- The goal state must be known.
- Searching backward from the goal may not always be easy.
- Both searches require memory to store their frontiers and explored states.

18. Compare BFS, UCS and DFS

Breadth-First Search (BFS), Uniform-Cost Search (UCS), and Depth-First Search (DFS) are uninformed search techniques used to find solutions in a search space. They differ in how they select the next node for expansion.

Comparison Table

Property	BFS	UCS	DFS
Search strategy	Expands the shallowest node first.	Expands the node with the lowest cumulative path cost.	Expands the deepest node first.

Data structure	FIFO queue	Priority queue	LIFO stack or recursion
Completeness	Yes, if the branching factor is finite.	Yes, if every step cost is at least some positive constant ε.	No, in infinite-depth spaces or spaces with loops; complete in finite spaces with appropriate cycle handling.
Optimality	Yes, if all step costs are equal.	Yes, for non-negative step costs, under the standard UCS conditions.	No, in general.

Property	**BFS**	**UCS**	**DFS**
Time complexity	$O(b^{d+1})$	\(O\left(b^{\lceil C^*/\varepsilon\rceil}\right)\)	$O(b^m)$
Space complexity	$O(b^{d+1})$	\(O\left(b^{\lceil C^*/\varepsilon\rceil}\right)\)	$O(bm)$
Memory usage	High	High	Low
Main advantage	Finds the shallowest solution.	Finds the least-cost solution.	Uses less memory.
Main disadvantage	Requires large memory.	Can require substantial time and memory.	May get stuck in infinite paths and is not optimal.

Notation

- b = branching factor.
- d = depth of the shallowest goal node.
- m = maximum depth of the search tree.
- C^* = cost of the optimal solution.
- ε = minimum positive step cost.

19. Consider the following search tree figure -2.4. Apply Iterative Deepening Search with depth limits 0, 1, 2 and 3. Show the nodes explored in each iteration and identify the iteration in which the goal is found.

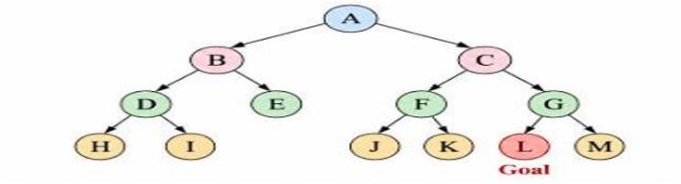


Figure 2.4

19. Apply Iterative Deepening Search (IDS) to Figure 2.4

Given: Depth limits 0,1,2, and 3.

Initial node: A

Goal node: L

Method: Iterative Deepening Search combines Depth-Limited Search (DLS) with a gradually increasing depth limit. The search follows a depth-first, left-to-right order in each iteration.

Iteration 1: Depth limit = 0

At depth limit 0, only the root node is explored.

A

Nodes explored: A

Result: Goal L is not found. Increase the depth limit to 1.

Iteration 2: Depth limit = 1

At depth limit 1, the search explores the root and its immediate children.

A

/ \

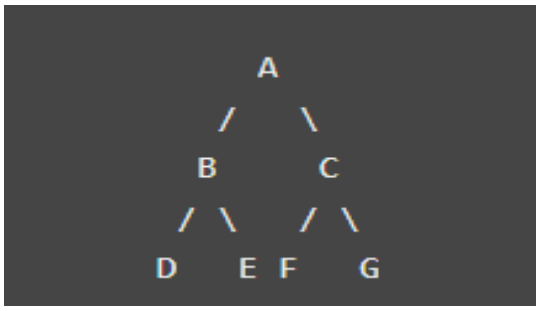
B C

Nodes explored: A → B → C

Result: Goal L is not found. Increase the depth limit to 2.

Iteration 3: Depth limit = 2

At depth limit 2, the search explores nodes up to level 2 using depth-first, left-to-right traversal.

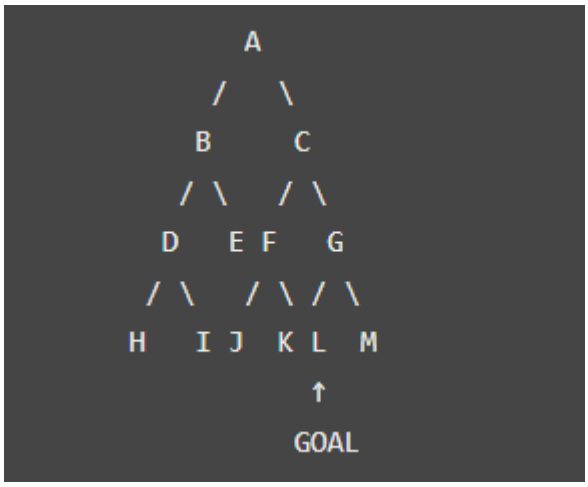


Nodes explored: A → B → D → E → C → F → G

Result: Goal L is not found because it is at depth 3. Increase the depth limit to 3.

Iteration 4: Depth limit = 3

At depth limit 3, the search explores the tree depth-first, from left to right, until the goal node L is found.



Nodes explored: A → B → D → H → I → E → C → F → J → K → G → L

Result: Goal L is found at depth 3. The search terminates.

Summary of Iterations

Iteration	Depth limit	Nodes explored	Goal found?

1	0	A	No
2	1	A, B, C	No
3	2	A, B, D, E, C, F, G	No
4	3	A, B, D, H, I, E, C, F, J, K, G, L	Yes

Final answer: The goal node L is found in the fourth iteration, when the depth limit reaches 3.

Key point: IDS repeats the search from the root at each iteration, increasing the depth limit by one until the goal is found.

20. Consider the following problem search trees. Apply BFS, DFS, Depth limit, and Iterative Deepening Search with depth limits 0, 1, 2 and 3.

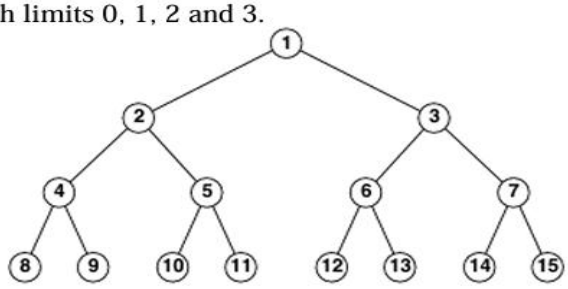


Figure-2.5: Here Initial Node=1 and Goal Node =11

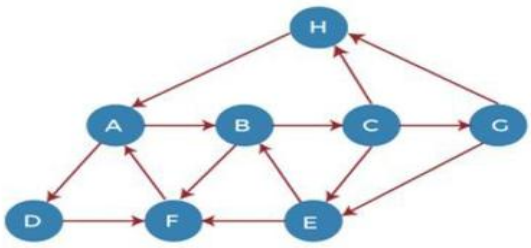


Figure-2.6:Here Initial Node=H and Goal Nodes= F and also E

20. Apply BFS, DFS, Depth-Limited Search and Iterative Deepening Search

The question contains two search graphs. We will solve both using the node connections and traversal order shown in the figures.

Figure 2.5 — Initial Node = 1, Goal Node = 11

The tree is traversed from left to right.

a) Breadth-First Search (BFS)

BFS explores all nodes at one level before moving to the next level.

Traversal:

$$1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11$$

The goal node 11 is found after exploring nodes 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 and 11.

Solution path:

$$1 \rightarrow 2 \rightarrow 5 \rightarrow 11$$

b) Depth-First Search (DFS)

DFS explores one branch as deeply as possible before backtracking.

Traversal:

$$1, 2, 4, 8, 9, 5, 10, 11$$

The search stops when node 11 is found.

Solution path:

$$1 \rightarrow 2 \rightarrow 5 \rightarrow 11$$

c) Depth-Limited Search (DLS)

DLS explores nodes depth-first but does not expand nodes beyond the specified depth limit.

Depth limit	Nodes explored	Goal 11 found?
0	1	No
1	1, 2, 3	No
2	1, 2, 4, 5, 3, 6, 7	No
3	1, 2, 4, 8, 9, 5, 10, 11	Yes

At depth limit 3, the goal node 11 is found.

d) Iterative Deepening Search (IDS)

IDS repeatedly performs DLS, increasing the depth limit by one in each iteration.

Iteration	Depth limit	Nodes explored	Goal found?
1	0	1	No
2	1	1, 2, 3	No
3	2	1, 2, 4, 5, 3, 6, 7	No
4	3	1, 2, 4, 8, 9, 5, 10, 11	Yes

Result: IDS finds node 11 in the fourth iteration, at depth 3.

Figure 2.6 — Initial Node = H, Goal Nodes = F and E

This is a directed graph. The arrows determine the valid direction of movement. We use left-to-right successor ordering and avoid revisiting already explored nodes.

a) Breadth-First Search (BFS)

BFS explores nodes level by level, starting from H.

Traversal:

$$\boxed{H, A, B, D, C, F}$$

The goal node F is found.

Solution path:

$$\boxed{H \rightarrow A \rightarrow D \rightarrow F}$$

b) Depth-First Search (DFS)

DFS explores one branch as deeply as possible before backtracking. Using the successor order B before D from A, the traversal is:

$$\boxed{H, A, B, C, G, E}$$

The goal node E is found.

Solution path:

$$\boxed{H \rightarrow A \rightarrow B \rightarrow C \rightarrow G \rightarrow E}$$

c) Depth-Limited Search (DLS)

Using the same DFS ordering, with the goal nodes F and E:

Depth limit	**Nodes explored**	**Goal found?**

0	H	No
1	H, A	No
2	H, A, B, D	No
3	H, A, B, C, D, F	Yes

At depth limit 3, the goal node F can be found through:

$$\boxed{H \rightarrow A \rightarrow D \rightarrow F}$$

d) Iterative Deepening Search (IDS)

IDS increases the depth limit gradually, repeating the depth-first search at each limit.

Iteration	**Depth limit**	**Nodes explored**	**Goal found?**
1	0	H	No
2	1	H, A	No
3	2	H, A, B, D	No
4	3	H, A, B, C, D, F	Yes

Result: IDS finds F at depth 3, through the path $H \rightarrow A \rightarrow D \rightarrow F$.